

Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias
Ingeniería en Ciencias de Datos y Matemáticas

Reporte Técnico: Casa Monarca

Integrantes del Equipo 5

Nombre completo	Matrícula
Alejandro Rojas	A00839067
Daniel DePool	A00839312
Luis Eduardo Martínez	A00840037
David Aguirre	A00839482
Arturo Camacho	A00838446

Grupo: 604

Profesores del bloque:

Dr. Raúl Gómez | Dr. Anas Wajid | Dr. Alberto F. Martínez

Socio Formador:

Casa Monarca: Ayuda Humanitaria y Apoyo al Migrante, A.C.

Monterrey, Nuevo León

11 de junio de 2026

Índice general

Índice de Tablas	IV
Índice de Figuras	V
Resumen	VI
1 Introducción	1
2 Marco de Referencia	3
2.1 Criptografía: Definición y Alcance	3
2.2 Criptografía de Clave Pública (Asimétrica)	3
2.2.1 RSA-2048	3
2.2.2 Certificados X.509 e Infraestructura de Clave Pública	3
2.3 Cifrado Simétrico Autenticado: AES-256-EAX	4
2.4 Derivación de Claves: Scrypt	4
2.5 Integridad de Datos: SHA-256 y Cadenas de Hash	4
2.6 Biblioteca Pycryptodome	4
2.7 Control de Acceso Basado en Roles (RBAC)	5
2.8 Sistemas Relacionados y Comparativa	5
3 Desarrollo	6
3.1 Propuesta de Solución	6
3.1.1 Descripción General de la Arquitectura	6
3.1.2 Justificación de Decisiones Técnicas	6
3.2 Esquema de la Base de Datos	7
3.3 Primitivas Criptográficas Implementadas	7
3.4 Los Dos Subsistemas de la Plataforma	8
3.4.1 Gestor de Identidades	8
3.4.2 Gestor de Expedientes con Acceso por Roles	8
3.5 Desarrollo de Componentes: Backend	9
3.5.1 Módulo Criptográfico Central (cripto/crypto.py)	9
3.5.2 Flujo de Login y Desbloqueo Criptográfico	11
3.5.3 Modelo de Usuario Extendido	11
3.5.4 Control de Acceso Granular a Expedientes	11
3.5.5 Bitácora de Auditoría con Encadenamiento de Hashes	12
3.5.6 Middleware de Expiración de Certificado	13
3.5.7 Portal del Migrante y Flujo ARCO	13
3.6 Desarrollo de Componentes: Frontend	14
3.7 Conexión de los Componentes	14
4 Resultados	15
4.1 Resultados del Backend	15
4.2 Resultados del Frontend	16
4.3 Solución Completa: Flujo de Extremo a Extremo	19
4.4 Casos de Uso y Tabla de Validaciones	19

5 Conclusiones y Trabajo Futuro	23
5.1 Conclusiones	23
5.2 Trabajo Futuro	23
Referencias	24
A Flujo Criptográfico Completo del Sistema	25
B Esquema Relacional de la Base de Datos	26

Índice de tablas

1	Integrantes del Equipo 5 y roles en el desarrollo	2
2	Fortaleza de clave en RSA comparado con algoritmos de clave privada	3
3	Campos criptográficamente relevantes del esquema de base de datos.	7
4	Primitivas criptográficas del sistema Casa Monarca	8
5	Acceso a expedientes según el rol del usuario	9
6	Roles del sistema y permisos asociados.	11
7	Tipos de eventos registrados en la bitácora de auditoría.	13
8	Casos de prueba y validaciones del sistema (elaboración propia).	19

Índice de figuras

1	Datos del expediente cifrados en la base de datos MySQL, ilegibles sin la llave AES (elaboración propia).	15
2	Bitácora de auditoría con encadenamiento de hashes SHA-256 (elaboración propia).	16
3	Pantalla de inicio de sesión del sistema	16
4	Dashboard adaptado al rol del usuario autenticado.	17
5	Lista de expedientes descifrados en tiempo real según el rol.	17
6	Panel de Administración con gestión de usuarios, roles y bitácora	18
7	Pantalla de ingreso de firma digital mediante archivo .key	18
8	Portal público del migrante para consulta de datos y derechos ARCO	19

Resumen

Este reporte documenta el diseño y la implementación de una plataforma de gestión segura de expedientes para Casa Monarca, organización que atiende a personas migrantes en situación de vulnerabilidad. El reto consiste en proteger datos personales altamente sensibles —nombres, nacionalidades, situaciones jurídicas e historiales psicosociales— de modo que ningún dato sea legible en reposo, incluso ante un acceso no autorizado a la base de datos.

La solución, construida con Django y MySQL, se basa en una **Arquitectura de Seguridad de Doble Llave**. La *Llave de Login* deriva una clave AES-256 desde la contraseña mediante Scrypt y descifra automáticamente la llave privada RSA-2048 del usuario al iniciar sesión. La *Llave de Firma*, inspirada en la e.firma del SAT, exige un certificado X.509 jerárquico y un archivo `.key` protegido por contraseña para las operaciones críticas (editar, eliminar, verificar y firmar expedientes).

Cada expediente se cifra con AES-256-EAX usando una llave simétrica única, y esa llave se distribuye cifrada con RSA-OAEP a cada rol autorizado, garantizando control de acceso granular. La bitácora de auditoría encadena sus registros con SHA-256, haciendo detectable cualquier manipulación. Un portal público permite al migrante consultar sus datos y ejercer derechos ARCO (Acceso, Rectificación, Cancelación y Oposición) usando solo su folio y nombre. Se diseñaron y ejecutaron 130 casos de prueba en ocho categorías funcionales, validando la completitud del modelo de seguridad implementado.

1. Introducción

Casa Monarca: Ayuda Humanitaria y Apoyo al Migrante, A.C. es una organización de la sociedad civil con sede en Monterrey, Nuevo León, que brinda atención integral a personas migrantes en situación de vulnerabilidad: asesoría legal, atención psicosocial, apoyo humanitario y gestión de documentación migratoria. En el ejercicio de esta labor, la organización recopila y almacena información de alta sensibilidad sobre los individuos que atiende, que incluye nombres completos, nacionalidades, fechas de nacimiento, datos de contacto, historiales médicos y psicológicos, así como situaciones jurídicas activas.

La problemática central es clara: los expedientes en su forma tradicional —ya sea en papel o en bases de datos digitales sin cifrado— representan un riesgo real de exposición para las personas atendidas. Una brecha de seguridad podría comprometer la integridad física o legal de migrantes en situación de persecución, estatus migratorio irregular o violencia de género. El reto planteado por el Socio Formador consiste en diseñar e implementar una plataforma que integra dos subsistemas complementarios:

1. **Gestor de Identidades:** administra a los colaboradores de Casa Monarca como identidades criptográficas. Cada usuario posee un par de llaves RSA-2048 y, en el caso de Administradores y Coordinadores, un certificado digital X.509 con su archivo de firma .key, equivalente a la e.firma del SAT. Este subsistema resuelve quién es cada usuario y qué puede hacer.
2. **Gestor de Expedientes con Acceso por Roles:** administra los expedientes de los migrantes, todos cifrados, y controla quién puede descifrarlos según su rol. El acceso no se concede a nivel de aplicación únicamente, sino a nivel criptográfico: solo quien posee la llave privada del rol correspondiente puede recuperar el contenido de un expediente. Este subsistema resuelve a qué información puede acceder cada usuario y bajo qué condiciones puede modificarla.

Ambos subsistemas se apoyan mutuamente: el Gestor de Identidades provee las llaves que el Gestor de Expedientes usa para cifrar y distribuir el acceso. En conjunto deben garantizar los siguientes objetivos:

- **Confidencialidad:** ningún dato personal de un migrante debe ser legible en reposo dentro de la base de datos, incluso ante acceso no autorizado a los servidores o al respaldo de la base de datos.
- **Control de acceso basado en roles (RBAC):** cada colaborador de Casa Monarca accede únicamente a la información que su rol le permite, siguiendo el principio de mínimo privilegio.
- **Integridad y firma digital:** toda modificación relevante a un expediente queda firmada criptográficamente por el responsable, garantizando la no repudiación.
- **Auditoría inviolable:** toda acción del sistema queda registrada en una bitácora cuya integridad es verificable, de forma que cualquier manipulación retroactiva sea detectable.
- **Derechos ARCO del migrante:** en cumplimiento con la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP), el sistema ofrece al migrante un mecanismo para ejercer sus derechos de Acceso, Rectificación, Cancelación y Oposición sobre sus propios datos, sin necesidad de cuenta de usuario.

La Etapa 1 de este reto cubre el diseño completo y la implementación funcional de la plataforma, incluyendo la arquitectura criptográfica, el backend en Django, el frontend web, el portal público del migrante, el sistema de auditoría, y una suite de 130 casos de prueba documentados. El sistema opera en red local (LAN), manteniendo los datos exclusivamente en servidores bajo control físico de Casa Monarca, sin dependencia de servicios en la nube de terceros. La Tabla 1 lista a los integrantes del equipo responsable del desarrollo.

Integrante	Rol en el equipo
Alejandro Rojas	Arquitectura criptográfica y backend
Daniel DePool	Backend y base de datos
Luis Eduardo Martínez	Backend y portal del migrante
David Aguirre	Frontend y casos de prueba
Arturo Camacho	Frontend y documentación técnica

Tabla 1: Integrantes del Equipo 5 y roles en el desarrollo .

2. Marco de Referencia

En este capítulo se describen los fundamentos teóricos, estándares internacionales y herramientas de la literatura que sustentan las decisiones de diseño tomadas por el equipo para la construcción del sistema de Casa Monarca.

2.1 Criptografía: Definición y Alcance

Jorge Ramió Aguirre define Criptografía como:

“La Criptografía es la rama inicial de las Matemáticas y en la actualidad también de la Informática y la Telemática, que hace uso de métodos y técnicas con el objeto principal de cifrar, y por tanto proteger, un mensaje o archivo por medio de un algoritmo, usando una o más claves” (Ramió Aguirre, 2006).

En el contexto de este proyecto, la criptografía no es un complemento opcional sino el núcleo arquitectónico de la solución. Cada campo sensible en la base de datos existe únicamente en forma cifrada; las claves de descifrado nunca se almacenan en claro y residen en memoria de sesión únicamente durante el tiempo necesario para ejecutar una operación.

2.2 Criptografía de Clave Pública (Asimétrica)

La criptografía asimétrica emplea un par de claves matemáticamente relacionadas: una pública, distribuable libremente, y una privada, que debe permanecer secreta. Cualquier dato cifrado con la clave pública solo puede ser descifrado con la clave privada correspondiente (Ramió Aguirre, 2006).

2.2.1 RSA-2048

El algoritmo RSA (Rivest–Shamir–Adleman) basa su seguridad en la dificultad computacional de factorizar el producto de dos números primos grandes. Con un módulo de 2048 bits, ofrece una seguridad equivalente a 112 bits de clave simétrica, nivel considerado adecuado para el horizonte temporal del presente proyecto según las recomendaciones del NIST (Barker y cols., 2019). La Tabla 2 ilustra la equivalencia entre tamaños de módulo RSA y seguridad simétrica.

Módulo clave pública (bits)	Equivalencia clave privada (bits)
2048	112
3072	128
4096	152
6144	176
8192	200

Tabla 2: Fortaleza de clave en RSA comparado con algoritmos de clave privada (tomado de Barker y cols. (2019)).

En el sistema implementado, RSA-2048 se usa con dos esquemas de relleno distintos según la operación: **PKCS1-OAEP** para el cifrado de llaves AES (encapsulamiento asimétrico de clave), y **PKCS#1 v1.5** con SHA-256 para la firma digital de expedientes y solicitudes ARCO. Esta distinción es relevante porque OAEP ofrece indistinguibilidad semántica (IND-CCA2) para cifrado, mientras que PKCS#1 v1.5 con SHA-256 es el estándar establecido para esquemas de firma (Barker y cols., 2019).

2.2.2 Certificados X.509 e Infraestructura de Clave Pública

Un certificado X.509 es un documento digital estándar (RFC 5280) que vincula una clave pública a una identidad mediante la firma de una Autoridad Certificadora (CA). En el sistema de Casa Monarca se

implementa una jerarquía de dos niveles, análoga a la e.firma del SAT mexicano:

- El **Administrador** posee un certificado auto-firmado que actúa como CA raíz de la organización.
- Los **Coordinadores** reciben certificados firmados por el Administrador emisor, estableciendo una cadena de confianza verificable criptográficamente.

Los certificados tienen vigencia de un año. La llave privada de firma se exporta en formato PKCS#8 cifrado con AES-256-CBC y Scrypt, formando el archivo `.key` que el usuario descarga una sola vez al momento de la creación de su cuenta.

2.3 Cifrado Simétrico Autenticado: AES-256-EAX

El Estándar de Cifrado Avanzado (AES, FIPS 197) es el cifrado de bloque más ampliamente adoptado en la industria. Con llave de 256 bits, ofrece 256 bits de seguridad simétrica, muy superior al mínimo recomendado de 128 bits para datos sensibles con horizonte temporal extendido.

El modo de operación EAX es un modo de cifrado autenticado con datos asociados (AEAD). A diferencia de modos como CBC o CTR, EAX produce simultáneamente el texto cifrado y un *tag* de autenticación de 128 bits que verifica tanto la confidencialidad como la integridad del mensaje. Si un atacante modifica cualquier bit del ciphertext, el tag de verificación fallará y el sistema rechazará el descifrado, haciendo imposibles los ataques de manipulación de datos en reposo (Barker y cols., 2019). Cada expediente utiliza un nonce único de 16 bytes generado criptográficamente con `os.urandom()`, eliminando ataques de análisis de patrones.

2.4 Derivación de Claves: Scrypt

La función de derivación de claves Scrypt fue diseñada para ser resistente a ataques con hardware especializado (GPU, FPGA, ASIC). Su diseño hace que el costo de un ataque de fuerza bruta escale tanto en tiempo de CPU como en memoria RAM, a diferencia de PBKDF2 que solo escala en tiempo de CPU.

Con los parámetros $N=16384$, $r=8$, $p=1$ y $dklen=32$, cada derivación requiere aproximadamente 16 MB de RAM y varios milisegundos de CPU. Scrypt se usa en dos contextos en el sistema: la derivación de la clave de login (desde contraseña más salt aleatorio único por usuario) y la derivación de la clave del migrante (desde folio más nombre completo, con salt determinístico igual a SHA-256(folio)).

2.5 Integridad de Datos: SHA-256 y Cadenas de Hash

Una de las herramientas fundamentales es la que Claude E. Shannon publicó en el artículo “A Mathematical Theory of Communication”, donde establece las bases de la llamada “Teoría de la Información” (Shannon, 2001). Dicha herramienta es el cálculo de la Entropía, que en términos generales mide la incertidumbre (aleatoriedad) de la información analizada. Sobre estos fundamentos se construye SHA-256, una función hash criptográfica que produce una huella digital de 256 bits con resistencia a preimagen, resistencia a segunda preimagen y resistencia a colisiones.

En el sistema, SHA-256 cumple tres roles distintos: (1) calcular el `hash_expediente` que forma parte de lo firmado digitalmente, de modo que la firma queda invalidada ante cualquier modificación del contenido; (2) calcular el `folio_hash` para búsqueda segura en el portal del migrante sin almacenar el folio en claro; y (3) encadenar los registros de la bitácora de auditoría de forma análoga a una cadena de bloques, donde el hash de cada evento depende del hash del evento anterior.

2.6 Biblioteca Pycryptodome

Pycryptodome es una biblioteca de Python de código abierto que provee implementaciones auditables de bajo nivel de primitivas criptográficas. Es el reemplazo mantenido activamente de PyCrypto, con soporte para RSA, AES en múltiples modos (incluyendo EAX), SHA-256, PKCS1-OAEP y PKCS#1 v1.5. El sistema importa los siguientes módulos: `Crypto.PublicKey.RSA` para generación e importación de pares RSA-2048; `Crypto.Cipher.AES` en modo EAX para cifrado autenticado; `Crypto.Cipher.PKCS1_OAEP` para encapsulamiento de llaves AES; `Crypto.Signature.pkcs1_15` para firma digital; y `Crypto.Hash.SHA256`

para el cómputo de hashes dentro de la firma. Complementariamente, la biblioteca `cryptography` (PyCA) se usa exclusivamente para la generación y parseo de certificados X.509, por su API de alto nivel.

2.7 Control de Acceso Basado en Roles (RBAC)

El modelo RBAC asigna permisos a roles y los roles a usuarios, en lugar de asignar permisos directamente a individuos. Esto simplifica la administración en organizaciones con muchos usuarios y permite aplicar el principio de mínimo privilegio. En el sistema, Django's Group y Permission implementan la capa de autorización de nivel aplicación, mientras que la distribución criptográfica de llaves RSA por rol constituye la capa de autorización de nivel criptográfico —más robusta, porque no puede ser eludida por manipulación de la aplicación sin poseer la llave privada correspondiente.

2.8 Sistemas Relacionados y Comparativa

Se revisaron las siguientes soluciones existentes antes de tomar las decisiones de diseño del sistema:

AWS KMS (Key Management Service): servicio gestionado en la nube para creación, rotación y auditoría de claves criptográficas. Se descartó porque implicaría transmitir datos de migrantes —potencialmente en situación irregular— a servidores de Amazon en territorio extranjero, lo que podría violar la LFPDPPP y representar un riesgo adicional para los individuos atendidos.

HashiCorp Vault: solución de gestión de secretos para entornos on-premise y nube. Más adecuada para secretos de infraestructura que para gestión de identidades de usuarios finales con firma digital. La complejidad operativa de despliegue y mantenimiento excede los recursos técnicos de Casa Monarca.

e.firma del SAT (México): el sistema del Servicio de Administración Tributaria implementa exactamente el modelo de firma digital con certificado X.509 más archivo `.key` cifrado que el equipo adoptó como referencia directa. Su familiaridad para usuarios mexicanos y su reconocimiento legal lo convierten en el estándar de facto replicado para el contexto de Casa Monarca.

La decisión de implementar una solución propia on-premise, usando exclusivamente bibliotecas de código abierto auditables (Pycryptodome, cryptography, Django), garantiza que Casa Monarca mantiene soberanía total sobre los datos de las personas que atiende, sin dependencia de terceros y sin costos de licenciamiento.

3. Desarrollo

3.1 Propuesta de Solución

3.1.1 Descripción General de la Arquitectura

La plataforma es una aplicación web monolítica construida sobre Django 4.2 (Python), con base de datos MySQL 8.0 y sin dependencias de servicios externos. El diseño central es la **Arquitectura de Seguridad de Doble Llave**, que opera en dos capas independientes de autenticación y autorización criptográfica:

Capa 1 — Llave de Login (transparente): al iniciar sesión con contraseña, el sistema deriva automáticamente una clave AES-256 mediante Scrypt y descifra la llave privada RSA-2048 del usuario, depositándola en la sesión de Django bajo la clave `_llave_privada_cache`. Simultáneamente, descifra las llaves privadas de los roles asignados al usuario y las deposita en `_llaves_rol_cache`. Este proceso es invisible para el usuario pero habilita el descifrado de expedientes durante toda la sesión activa.

Capa 2 — Llave de Firma (explícita): para operaciones críticas —editar, eliminar o verificar expedientes, y toda acción en el Panel de Administración— el sistema requiere que el usuario suba su archivo `.key` e ingrese su passphrase de 64 caracteres hexadecimales. La llave privada de firma se mantiene en sesión por 15 minutos. Pasado ese tiempo, cualquier operación crítica solicita nuevamente la llave, siguiendo el mismo modelo de ventana temporal que implementa el SAT en su sistema de e.firma.

3.1.2 Justificación de Decisiones Técnicas

Django sobre Flask/FastAPI: Django provee de forma nativa el ORM, el sistema de autenticación, el motor de templates y el framework de permisos y grupos. Flask requeriría integrar extensiones separadas para cada uno, aumentando la complejidad y la superficie de ataque. FastAPI está orientado a APIs REST, lo que no se alinea con el perfil de aplicación web con sesiones y templates necesario para Casa Monarca.

MySQL sobre PostgreSQL o SQLite: MySQL 8.0 es el motor más familiar para el equipo técnico de Casa Monarca y el más común en hostings compartidos de bajo costo disponibles en México. SQLite no soporta la concurrencia adecuada para un sistema multiusuario. PostgreSQL es igualmente válido, pero no ofrece ventajas decisivas para este perfil de uso.

Hosting local (LAN) sobre servicios cloud: los datos de migrantes son de carácter sensible y potencialmente crítico para la seguridad personal de los individuos. Mantenerlos en servidores bajo control físico de Casa Monarca elimina el riesgo de exposición ante violaciones de datos en proveedores de nube y cumple con el principio de minimización de datos de la LFPDPPP.

RSA-2048 sobre RSA-4096: RSA-4096 ofrece mayor seguridad a largo plazo pero cuadruplica el tiempo de operación de cifrado y descifrado. Para el volumen de operaciones de Casa Monarca y el horizonte temporal del proyecto, RSA-2048 (equivalente a 112 bits simétricos) es suficiente según las guías del NIST (Barker y cols., 2019).

AES-256-EAX sobre AES-CBC o AES-GCM: EAX provee cifrado autenticado (AEAD), por lo que el tag de autenticación detecta automáticamente cualquier manipulación del ciphertext. CBC no ofrece autenticación integrada (requiere un HMAC separado), lo que abre vulnerabilidades como ataques de relleno (padding oracle). GCM es igualmente válido pero requiere mayor cuidado en la gestión del nonce; EAX tiene garantías de seguridad más robustas en caso de reutilización accidental de nonce.

Scrypt sobre PBKDF2: PBKDF2 es resistente a ataques de tiempo pero no a ataques con hardware especializado (GPU/ASIC). Scrypt requiere tanto tiempo de CPU como memoria RAM proporcional al parámetro N , haciendo que ataques paralelos con GPU sean significativamente más costosos, protegiendo contraseñas débiles con mayor margen de seguridad.

3.2 Esquema de la Base de Datos

La base de datos MySQL contiene ocho tablas principales. La Tabla 3 presenta los campos criptográficamente relevantes de cada tabla.

Tabla / Campo	Cifrado	Descripción
<i>usuarios_usuario</i>		
llave_privada	AES-EAX(SECRET_KEY) + AES-EAX(Scrypt(pwd))	Doble cifrado: capa BD + capa login
telefono	AES-EAX(SECRET_KEY)	Campo transparente EncryptedCharField
certificado_digital	Sin cifrar (PEM público)	Certificado X.509 del usuario
<i>usuarios_accesoollaverol</i>		
llave_privada_rol_cifrada	RSA-OAEP(llave_pub_usuario)	Llave privada del rol cifrada con llave del usuario
<i>expediente_expediente</i>		
datos_cifrados	AES-256-EAX(llave_aes)	Todos los datos personales del migrante
firma_digital	RSA-2048-PKCS1	Firma sobre SHA-256 del ciphertext
folio_hash	SHA-256(folio)	Índice de búsqueda segura en portal
etiquetas_oposicion	AES-EAX(SECRET_KEY)	JSON de oposiciones ARCO
<i>expediente_accesoexpediente</i>		
llave_aes_cifrada	RSA-OAEP(llave_pub_rol/usuario)	Una fila por cada entidad con acceso
<i>portal_migrante_solicitudarco</i>		
descripcion_cifrada	AES-EAX(SECRET_KEY)	Texto libre del migrante
cambios_propuestos	AES-EAX(SECRET_KEY)	Diff propuesto por el Operativo
firma_digital	RSA-2048-PKCS1	Firma del Coordinador sobre la solicitud
firma_digital_admin	RSA-2048-PKCS1	Firma del Admin en cancelaciones
<i>auditoria_bitacoraevento</i>		
hash_registro	SHA-256 encadenado	Hash del evento ligado al anterior

Tabla 3: Campos criptográficamente relevantes del esquema de base de datos.

3.3 Primitivas Criptográficas Implementadas

La Tabla 4 resume los algoritmos implementados en el módulo central `cripto/crypto.py`.

Algoritmo	Parámetros	Uso en el sistema
RSA-2048	PKCS1-OAEP / PKCS#1 v1.5	Par de llaves por usuario y por rol; encapsulamiento de llaves AES; firma digital
AES-256-EAX	Nonce 16 B, Tag 16 B	Cifrado autenticado de expedientes y llaves privadas en reposo
Scrypt	N=16384, r=8, p=1, dklen=32	Derivación de clave desde contraseña (login) y desde folio+nombre (portal migrante)
SHA-256	Hex 64 chars	Hash de expedientes; folio_hash; encadenamiento de bitácora
X.509	1 año de vigencia	Certificado jerárquico (auto-firmado Admin; firmado por Admin para Coordinadores)
PKCS#8	AES-256-CBC + Scrypt	Exportación protegida de llave privada al archivo .key
AES-EAX (BD)	SHA-256(SECRET_KEY)	Cifrado transparente de campos sensibles en base de datos

Tabla 4: Primitivas criptográficas del sistema Casa Monarca .

3.4 Los Dos Subsistemas de la Plataforma

La plataforma integra dos subsistemas que operan de forma coordinada pero con responsabilidades distintas: el Gestor de Identidades y el Gestor de Expedientes con Acceso por Roles. Entender la separación entre ambos es clave para comprender el diseño de seguridad completo.

3.4.1 Gestor de Identidades

El Gestor de Identidades es responsable de crear, administrar y revocar las identidades criptográficas de los colaboradores de Casa Monarca. Cada vez que un Administrador da de alta a un usuario, el sistema genera automáticamente un par de llaves RSA-2048 exclusivo para esa persona. La llave pública se almacena en claro (es información no sensible), mientras que la llave privada se cifra en doble capa antes de guardarse en la base de datos, como se describe en la subsección del modelo de usuario.

Para los roles que requieren firmar documentos —Administrador y los cinco tipos de Coordinador— el Gestor de Identidades emite además un certificado digital X.509 y un archivo .key protegido por una contraseña hexadecimal de 64 caracteres. El certificado del Administrador es auto-firmado y actúa como Autoridad Certificadora raíz de la organización; los certificados de los Coordinadores son firmados por el Administrador, estableciendo una cadena de confianza verificable. Este diseño replica fielmente el modelo de la e.firma del SAT, donde la identidad digital se compone de un certificado público y una llave privada protegida que el titular custodia.

3.4.2 Gestor de Expedientes con Acceso por Roles

El Gestor de Expedientes administra los registros de los migrantes y, lo más importante, controla quién puede leerlos y modificarlos según su rol. El mecanismo de acceso opera en dos niveles complementarios:

Nivel de aplicación (Django RBAC): mediante grupos y permisos de Django, el sistema decide qué botones, vistas y acciones están disponibles para cada rol. Por ejemplo, el botón “Eliminar expediente” solo aparece para el Administrador, y la acción “Verificar” solo para Coordinadores con firma activa. Este nivel es la primera barrera y mejora la experiencia de usuario, pero por sí solo no garantiza confidencialidad, pues podría ser eludido manipulando la aplicación.

Nivel criptográfico (distribución de llaves por rol): esta es la barrera fundamental. Cuando se crea un expediente, sus datos se cifran con una llave AES-256 única. Esa llave AES no se guarda en claro: se distribuye cifrada con RSA-OAEP usando la llave pública de cada rol autorizado. De este modo, un Coordinador solo puede descifrar un expediente si posee la llave privada de su rol, la cual a su vez solo se

desbloquea con su contraseña al iniciar sesión. Aunque un atacante eludiera el control de la aplicación, sin la llave privada del rol obtendría únicamente texto cifrado ilegible. Este es el corazón del control de acceso por roles: el acceso está respaldado por matemáticas, no solo por reglas de software.

La diferencia entre roles se refleja directamente en las llaves que cada usuario posee en su sesión. Un Operativo solo tiene la llave privada del rol Operativo, por lo que solo descifra los expedientes a los que el rol Operativo tiene acceso. Un Administrador posee todas las llaves de rol del sistema, por lo que puede descifrar cualquier expediente. La Tabla 5 resume el acceso a expedientes por rol.

Rol	Llaves privadas en sesión	Alcance sobre expedientes
Administrador	Todas las llaves de rol	Lee y descifra todos los expedientes; único que puede eliminar
Coordinador (5 tipos)	Llave de su propio rol	Lee, edita, verifica y firma los expedientes accesibles a su rol
Operativo	Llave del rol Operativo	Lee los expedientes accesibles a Operativo; crea nuevos registros
Usuario	Llave personal (sin llave de rol)	Solo ve los expedientes que él mismo creó
Migrante	Clave derivada de folio+nombre	Solo su propio expediente, vía portal público

Tabla 5: Acceso a expedientes según el rol del usuario .

3.5 Desarrollo de Componentes: Backend

3.5.1 Módulo Criptográfico Central (cripto/crypto.py)

El módulo `cripto/crypto.py` encapsula la totalidad de las operaciones criptográficas del sistema en funciones puras, sin efectos secundarios sobre la base de datos. Sus funciones se organizan en seis grupos: (1) generación y manejo de pares RSA-2048, (2) generación de certificados X.509, (3) derivación de claves de login con Scrypt, (4) cifrado y descifrado AES-256-EAX, (5) hashing SHA-256 y firma digital, y (6) cifrado transparente de base de datos y acceso del portal del migrante.

El Código 3.1 muestra la función de derivación de clave para el flujo de login.

```

1 def derivar_clave_login(password: str, salt_hex: str) -> bytes:
2     """Deriva una clave AES-256 con scrypt desde el password y salt."""
3     salt_bytes = bytes.fromhex(salt_hex)
4     return hashlib.scrypt(
5         password.encode('utf-8'),
6         salt=salt_bytes,
7         n=16384, # Factor de costo CPU/memoria
8         r=8,     # Tamaño de bloque
9         p=1,     # Paralelismo
10        dklen=32  # 256 bits = llave AES-256
11    )

```

Listing 3.1: Derivación de clave Scrypt para autenticación de usuarios.

El Código 3.2 muestra el cifrado AES-256-EAX de un expediente.

```

1 def cifrar_datos_sin_rsa(datos: dict) -> dict:
2     """
3     Cifra un diccionario con AES-256-EAX.
4     Retorna datos_cifrados, nonce, tag y la llave AES en crudo.
5     La llave AES NO se cifra aqui; el llamante la cifra con RSA
6     para cada destinatario (creador + roles).
7     """
8     llave_aes = os.urandom(32) # 256 bits aleatorios criptograficamente seguros
9     datos_json = json.dumps(datos, ensure_ascii=False).encode('utf-8')
10    cipher = AES.new(llave_aes, AES.MODE_EAX)
11    datos_cifrados, tag = cipher.encrypt_and_digest(datos_json)
12    return {
13        'datos_cifrados': base64.b64encode(datos_cifrados).decode('utf-8'),
14        'nonce':          base64.b64encode(cipher.nonce).decode('utf-8'),

```

```

15         'tag':          base64.b64encode(tag).decode('utf-8'),
16         'llave_aes':    llave_aes,      # Descartado de RAM tras distribucion RSA
17     }

```

Listing 3.2: Cifrado AES-256-EAX de datos del expediente.

El Código 3.3 muestra la firma digital y su verificación.

```

1  def firmar(datos: str, llave_privada_pem: str) -> str:
2      """Firma un string con RSA-2048 + SHA-256. Retorna Base64."""
3      priv_key = RSA.import_key(llave_privada_pem)
4      h = SHA256.new(datos.encode('utf-8'))
5      firma = pkcs1_15.new(priv_key).sign(h)
6      return base64.b64encode(firma).decode('utf-8')
7
8  def verificar_firma(datos: str, firma_b64: str, llave_publica_pem: str) -> bool:
9      """Verifica una firma digital. Retorna True si es valida."""
10     try:
11         pub_key = RSA.import_key(llave_publica_pem)
12         h = SHA256.new(datos.encode('utf-8'))
13         pkcs1_15.new(pub_key).verify(h, base64.b64decode(firma_b64))
14         return True
15     except (ValueError, TypeError):
16         return False

```

Listing 3.3: Firma digital RSA-2048 con SHA-256 y verificación.

El Código 3.4 muestra la generación del certificado X.509 jerárquico.

```

1  def generar_certificado(privada_pem, publica_pem, username,
2                          passphrase=None, issuer_privada_pem=None,
3                          issuer_username=None, auto_firmado=False):
4      """Genera certificado X.509. Si se proporciona issuer_privada_pem,
5      el certificado es firmado por el emisor (Coordinador firmado por Admin).
6      Si auto_firmado=True, el Admin firma su propio certificado (CA raiz).
7      Retorna (cert_pem, cert_der, expiracion)."""
8      pwd = passphrase.encode('utf-8') if passphrase else None
9      private_key = serialization.load_pem_private_key(
10         privada_pem.encode('utf-8'), password=pwd)
11     public_key = serialization.load_pem_public_key(publica_pem.encode('utf-8'))
12     subject = x509.Name([
13         x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"Casa Monarca"),
14         x509.NameAttribute(NameOID.COMMON_NAME, username),
15     ])
16     if issuer_privada_pem and issuer_username:
17         issuer = x509.Name([
18             x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"Casa Monarca"),
19             x509.NameAttribute(NameOID.COMMON_NAME, issuer_username),
20         ])
21         signing_key = serialization.load_pem_private_key(
22             issuer_privada_pem.encode('utf-8'), password=None)
23     elif auto_firmado:
24         issuer = subject
25         signing_key = private_key
26     else:
27         raise ValueError("Se requiere llave de emisor o auto_firmado=True.")
28     ahora = datetime.datetime.utcnow()
29     expiracion = ahora + datetime.timedelta(days=365)
30     cert = (x509.CertificateBuilder()
31         .subject_name(subject).issuer_name(issuer)
32         .public_key(public_key).serial_number(x509.random_serial_number())
33         .not_valid_before(ahora).not_valid_after(expiracion)
34         .sign(signing_key, hashes.SHA256()))
35     return (cert.public_bytes(serialization.Encoding.PEM).decode('utf-8'),
36         cert.public_bytes(serialization.Encoding.DER), expiracion)

```

Listing 3.4: Generación de certificado X.509 jerárquico estilo SAT.

3.5.2 Flujo de Login y Desbloqueo Criptográfico

Al autenticarse, el sistema no solo valida la contraseña con Django (PBKDF2): además deriva la clave Script y descifra la llave privada RSA del usuario para depositarla en la sesión. El Código 3.5 muestra la lógica de desbloqueo automático dentro de usuarios/views.py.

```

1  if user.activo:
2      login(request, user)
3      # Desbloqueo automatico: Llave de Login
4      if user.llave_privada and user.salt_login:
5          try:
6              llave_privada_pem = descifrar_llave_con_password(
7                  user.llave_privada, password, user.salt_login
8              )
9              request.session['_llave_privada_cache'] = llave_privada_pem
10             # Cargar llaves privadas de cada rol asignado al usuario
11             llaves_rol_cache = {}
12             for acceso in user.accesos_rol.select_related('llave_rol').all():
13                 paquete = json.loads(acceso.llave_privada_rol_cifrada)
14                 datos = descifrar_datos(paquete, llave_privada_pem)
15                 llaves_rol_cache[acceso.llave_rol.rol] = datos['key']
16             request.session['_llaves_rol_cache'] = llaves_rol_cache
17         except ValueError:
18             pass # Sesion cripto vacia; no puede descifrar expedientes
19     # Registrar en bitacora de auditoria
20     BitacoraEvento.objects.create(
21         usuario=user, tipo='login',
22         descripcion=f'Inicio de sesion de {user.username}',
23         ip=_get_client_ip(request),
24     )

```

Listing 3.5: Desbloqueo automático de llaves criptográficas en el login.

3.5.3 Modelo de Usuario Extendido

El modelo Usuario en usuarios/models.py extiende AbstractUser de Django con campos criptográficos adicionales. Los campos llave_privada y telefono utilizan tipos de campo personalizados (EncryptedTextField y EncryptedCharField) que sobrescriben los métodos get_prep_value y from_db_value de Django para cifrar automáticamente al escribir y descifrar al leer. La llave privada RSA sufre doble cifrado: la capa de base de datos (AES-EAX con SECRET_KEY) envuelve la capa de contraseña (AES-EAX con Script(password, salt)). Solo un atacante que comprometiera simultáneamente el SECRET_KEY del servidor y la contraseña del usuario podría descifrar la llave privada.

Los ocho roles del sistema y sus permisos se detallan en la Tabla 6.

Rol	Permisos
Administrador	Crear, ver, editar, eliminar, exportar expedientes; ver bitácora; gestionar usuarios; acceso a todas las llaves de rol
Coordinador_Administracion	Crear, ver, editar, exportar expedientes; firmar y verificar; acceso a llave de rol propio
Coordinador_Legal	Crear, ver, editar, exportar expedientes; firmar y verificar
Coordinador_Psicosocial	Crear, ver, editar, exportar expedientes; firmar y verificar
Coordinador_Humanitario	Crear, ver, editar, exportar expedientes; firmar y verificar
Coordinador_Comunicacion	Crear, ver, editar, exportar expedientes; firmar y verificar
Operativo	Crear y ver expedientes; aprobar solicitudes ARCO
Usuario	Crear expedientes (solo registro inicial)

Tabla 6: Roles del sistema y permisos asociados.

3.5.4 Control de Acceso Granular a Expedientes

El control de acceso granular se materializa en dos momentos: la *creación* (distribución de la llave AES a los roles) y la *lectura* (descifrado selectivo según las llaves que el usuario posee en sesión).

Al crear un expediente, el sistema genera una llave AES-256 aleatoria, cifra los datos y crea una fila AccesoExpediente por cada entidad que debe poder descifrar el expediente: el usuario creador y los siete roles de destino. Cada fila contiene la misma llave AES cifrada con la llave pública del destinatario diferente. El Código 3.6 muestra esta distribución en expediente/views.py.

```

1 ROLES_DESTINO_EXPEDIENTE = [
2     'Administrador', 'Coordinador_Administracion', 'Coordinador_Legal',
3     'Coordinador_Psicosocial', 'Coordinador_Humanitario',
4     'Coordinador_Comunicacion', 'Operativo',
5 ]
6
7 def _crear_accesos_expediente(expediente, llave_aes_bytes, usuario):
8     # Copia para el creador (llave publica personal del usuario)
9     AccesoExpediente.objects.create(
10         expediente=expediente, tipo_acceso='Creador', usuario=usuario,
11         llave_aes_cifrada=cifrar_llave_aes(llave_aes_bytes, usuario.llave_publica),
12     )
13     # Copia para cada rol con su llave publica RSA
14     for rol_name in ROLES_DESTINO_EXPEDIENTE:
15         pub_rol = _obtener_llave_rol_publica(rol_name)
16         if pub_rol:
17             AccesoExpediente.objects.create(
18                 expediente=expediente, tipo_acceso=rol_name,
19                 llave_aes_cifrada=cifrar_llave_aes(llave_aes_bytes, pub_rol),
20             )

```

Listing 3.6: Distribución de acceso al expediente mediante RSA-OAEP por rol.

En la *lectura*, cuando un usuario abre la lista de expedientes, el sistema recorre cada expediente e intenta descifrarlo usando las llaves de rol que el usuario cargó en su sesión al iniciar sesión. Para cada expediente busca la fila AccesoExpediente correspondiente a alguno de los roles del usuario, descifra la llave AES con RSA-OAEP usando la llave privada de ese rol, y con la llave AES recuperada descifra los datos del expediente. Si el usuario no posee ninguna llave de rol con acceso a ese expediente, el sistema no muestra el contenido sino la leyenda “Datos cifrados (sin acceso)”. De esta forma, el mismo listado muestra contenido distinto según el rol de quien lo consulta, sin que exista ningún punto donde los datos se expongan a un usuario no autorizado. El Código 3.7 ilustra este descifrado selectivo.

```

1 def _descifrar_expediente_para_usuario(expediente, llaves_rol_cache):
2     """Intenta descifrar el expediente con las llaves de rol del usuario.
3     Retorna el dict de datos si tiene acceso, o None si no lo tiene."""
4     for rol, llave_privada_rol in llaves_rol_cache.items():
5         acceso = expediente.accesos.filter(tipo_acceso=rol).first()
6         if not acceso:
7             continue
8         try:
9             llave_aes = descifrar_llave_aes(
10                 acceso.llave_aes_cifrada, llave_privada_rol)
11             return descifrar_datos_con_aes_existente(
12                 expediente.datos_cifrados, expediente.nonce,
13                 expediente.tag, llave_aes)
14         except (ValueError, TypeError):
15             continue # Esta llave de rol no abre este expediente; probar otra
16     return None # Sin acceso: la vista mostrara "Datos cifrados"

```

Listing 3.7: Descifrado selectivo de expediente según las llaves de rol en sesión.

3.5.5 Bitácora de Auditoría con Encadenamiento de Hashes

El modelo BitacoraEvento en auditoria/models.py sobrescribe el método save() para calcular automáticamente el hash de cada evento encadenado con el hash del evento anterior, según la ecuación:

$$h_N = \text{SHA-256}(h_{N-1} \parallel \text{usuario_id} \parallel \text{tipo} \parallel \text{descripcion} \parallel \text{fecha}) \quad (3.1)$$

El primer evento de la cadena usa el literal 'GENESIS' como h_0 . Esta estructura garantiza que cualquier modificación, eliminación o reordenamiento de registros sea detectable al recalculando la cadena, pues rompe

la consistencia de todos los hashes posteriores al registro manipulado. La Tabla 7 lista los tipos de eventos registrados.

Tipo de evento	Cuándo se registra
login / logout	Inicio y cierre de sesión
cambio_rol	Cambio de rol de usuario por el Administrador
toggle_permiso	Modificación de permiso individual
toggle_activo	Activación o desactivación de cuenta
solicitud_creada/aprobada/rechazada	Ciclo de vida de solicitudes de cambio de rol
verificacion_expediente	Edición o verificación de expediente
eliminar_expediente	Eliminación de expediente
acceso_migrante	Acceso exitoso al portal público
acceso_migrante_fallido	Intento fallido o IP bloqueada en portal
solicitud_arco_creada	Solicitud ARCO enviada por el migrante
solicitud_arco_firmada	Firma criptográfica de solicitud ARCO
solicitud_arco_ejecutada	Ejecución de rectificación u oposición
expediente_cancelado_arco	Eliminación física por cancelación ARCO
aviso_privacidad_aceptado	Aceptación del Aviso de Privacidad al registrar

Tabla 7: Tipos de eventos registrados en la bitácora de auditoría.

3.5.6 Middleware de Expiración de Certificado

El CertificadoExpiracionMiddleware en `cripto/middleware.py` intercepta cada solicitud HTTP de usuarios con rol Administrador o Coordinador. Verifica que el usuario tenga llave pública, llave privada y certificado X.509 vigente. Si alguna condición falla, ejecuta un logout forzado y redirige al login con un mensaje de error. Los roles Usuario y Operativo están exentos por no requerir certificado de firma. Las rutas del portal del migrante (`/mi-expediente/`) también están excluidas, ya que los migrantes no son usuarios Django autenticados.

3.5.7 Portal del Migrante y Flujo ARCO

El portal público en `/mi-expediente/` permite al migrante acceder a sus datos usando su folio (CM-YYYYMMDD-XXXX) y su nombre completo. La autenticación es puramente criptográfica: el sistema deriva una clave Script del par (folio, nombre), la usa para descifrar la llave AES almacenada en `AccesoExpediente` con `tipo_acceso='Migrante'`, y con esa llave descifra el expediente completo. Un rate limit de cinco intentos por IP cada 15 minutos protege contra ataques de fuerza bruta. El portal implementa tres flujos ARCO:

- **Rectificación:** el migrante solicita corrección de sus datos. El Operativo propone los cambios concretos (diff JSON). El Coordinador firma criptográficamente con su `.key` y aplica el diff, re-cifrando el expediente con la misma llave AES y actualizando la firma digital y el hash.
- **Oposición:** el migrante solicita restricción de uso de sus datos. El Operativo escribe una etiqueta de oposición. El Coordinador firma y la anexa al campo cifrado `etiquetas_oposicion` del expediente.
- **Cancelación:** requiere cuatro actores en cinco estados: migrante (solicita), Operativo (pre-aprueba), Coordinador (firma validación), Administrador (firma y ejecuta la eliminación física). El expediente se borra de la BD, pero la `SolicitudARCO` conserva el `hash_expediente` previo y las firmas como registro histórico auditado, sin datos personales identificables.

El Código 3.8 muestra la derivación de clave y acceso del migrante.

```

1 def derivar_clave_folio(folio: str, nombre_completo: str) -> bytes:
2     """
3     Deriva clave AES-256 desde folio + nombre.
4     Doble factor: 'algo que tiene' (folio) + 'algo que es' (nombre).
5     Salt = SHA-256(folio): cada derivación es independiente.
6     """
7     nombre_normalizado = nombre_completo.strip().lower()
8     secreto = f"{folio}:{nombre_normalizado}".encode('utf-8')
9     salt = hashlib.sha256(folio.encode('utf-8')).digest()

```

```
10 | return hashlib.scrypt(secreto, salt=salt, n=16384, r=8, p=1, dklen=32)
```

Listing 3.8: Derivación de clave Scrypt para acceso del migrante por folio.

3.6 Desarrollo de Componentes: Frontend

El frontend está construido íntegramente con Django Templates (Jinja2), HTML5, CSS3 (Bootstrap 5) y JavaScript vanilla. No se utiliza ningún framework de frontend separado, manteniendo la arquitectura simple y manejable para el equipo de Casa Monarca. Todas las páginas comparten el template base `cm_base.html` que incluye el encabezado con el logo de Casa Monarca, la barra de navegación con el rol actual del usuario, indicador de estado de llaves y certificado, y pie de página.

Las vistas principales implementadas son:

- **Login** (`usuarios/login.html`): formulario de autenticación con validación de cuenta desactivada y mensajes de error genéricos.
- **Dashboard** (`expediente/dashboard.html`): resumen de permisos del rol, accesos rápidos a funciones, badges de notificación para solicitudes pendientes (roles y ARCO), y estado visual de llaves y certificado.
- **Lista de Expedientes** (`expediente/lista_expedientes.html`): tabla con todos los expedientes descifrados en tiempo real, filtros por estado de verificación, creador y rango de fechas, columnas de acciones condicionadas al rol (editar solo Coordinador/Admin; eliminar solo Admin; verificar solo con firma activa).
- **Panel de Administración** (`usuarios/admin_panel.html`): cuatro pestañas — Usuarios (gestión completa con badges de seguridad RSA/X.509), Solicitudes de Rol, Bitácora (últimos 20 eventos con iconos por tipo) y Criptografía (regenerar identidad, revocar certificado, restablecer contraseña).
- **Ingresar Firma** (`usuarios/ingresar_firma.html`): formulario para subir el archivo `.key` y la passphrase. Valida que el módulo de la llave privada coincida con el del certificado X.509 registrado para el usuario.
- **Portal del Migrante** (`portal_migrante/`): interfaz pública con aviso de privacidad, vista de datos personales en modo solo lectura, y formularios de solicitudes ARCO con seguimiento de estado.

3.7 Conexión de los Componentes

La configuración de URLs en `config/urls.py` organiza las rutas en tres namespaces: `usuarios/` para autenticación y gestión de cuentas, `expediente/` para el CRUD de expedientes y solicitudes ARCO, y `mi-expediente/` para el portal del migrante. El middleware de certificados actúa antes de que cualquier vista procese la solicitud HTTP. Los decoradores `@rol_requerido` y `@firma_requerida` en `usuarios/decorators.py` aplican control de acceso a nivel de función de vista. La `SECRET_KEY` de Django se lee desde variable de entorno (`.env`) y se usa tanto para el cifrado transparente de campos de BD como para la firma de cookies de sesión. Las llaves criptográficas del usuario se almacenan en la sesión del servidor (base de datos de sesiones de Django), nunca en la cookie del cliente.

4. Resultados

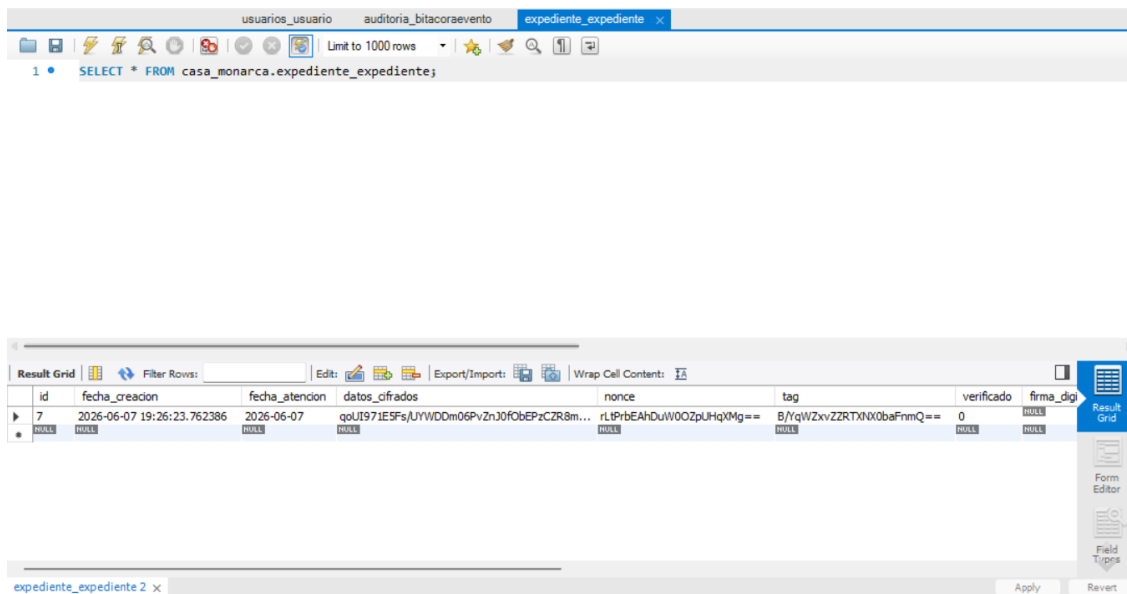
4.1 Resultados del Backend

El módulo `cripto/crypto.py` implementa correctamente todas las primitivas criptográficas requeridas. Las funciones operan simétricamente: todo lo cifrado con `cifrar_datos_sin_rsa` se recupera exactamente con `descifrar_datos_con_aes_existente`; toda firma producida con `firmar` es verificable con `verificar_firma`; los certificados generados con `generar_certificado` son parseable con las bibliotecas estándar X.509. La generación de identidad criptográfica completa para un nuevo Coordinador —par RSA-2048, certificado X.509 firmado por el Admin, archivo `.key` cifrado con passphrase de 64 caracteres hexadecimales, y ZIP descargable— opera correctamente en un único flujo de creación de usuario.

El flujo de login desbloquea correctamente la llave privada del usuario y carga las llaves de rol en la sesión, habilitando el descifrado de expedientes de forma transparente. El sistema maneja correctamente el caso donde la llave privada está corrupta o el salt es inválido: el login Django procede pero la sesión criptográfica queda vacía (sin `_llave_privada_cache`), impidiendo el descifrado de expedientes sin comprometer la autenticación de la aplicación.

La bitácora de auditoría genera correctamente la cadena de hashes SHA-256 desde el evento génesis. Al modificar artificialmente un registro intermedio en la base de datos y recalculer la cadena, el sistema detecta la discrepancia en el registro alterado y en todos los posteriores, confirmando la propiedad de integridad encadenada.

En la Fig. 1 se muestra el resultado de la prueba TC-08-01: al inspeccionar directamente la tabla `expediente_expediente` en MySQL, los datos personales del migrante aparecen únicamente como texto cifrado en Base64, ilegibles sin la llave AES correspondiente. Esto confirma que la confidencialidad en reposo se cumple incluso ante acceso directo a la base de datos.



id	fecha_creacion	fecha_atencion	datos_cifrados	nonce	tag	verificado	firma_digi
7	2026-06-07 19:26:23.762386	2026-06-07	qoU1971ESFs/JYWD0m06PvZn30F0bEPzCZR8m...	rLpPbEAH0u/W00ZpUHQXmg==	B/YqWZxvZZRTX0X0baFrmQ==	0	

Figura 1: Datos del expediente cifrados en la base de datos MySQL, ilegibles sin la llave AES (elaboración propia).

En la Fig. 2 se observa la bitácora de auditoría, donde cada evento almacena su propio hash SHA-256 encadenado con el del evento anterior, formando una estructura análoga a una cadena de bloques que hace detectable cualquier manipulación retroactiva.

id	tipo	descripcion	fecha	ip	hash_registro	usuario_id
1	login	Inicio de sesión de admin_contingencia	2026-05-14 19:28:11.718304	127.0.0.1	a2304860249de3708f9c0059a70a9d22166548...	2
2	cambio_rol	Creó nuevo usuario: Alex con rol Coordinador_L...	2026-05-15 03:11:00.870985	127.0.0.1	6105926d7ad1b6d659b9b7619f61fb40f7cb93...	2
3	logout	Cierre de sesión de admin_contingencia	2026-05-15 03:12:33.570095	127.0.0.1	14cdc56191cc6d1d50718229d636ede9df73020...	2
4	login	Inicio de sesión de Alex	2026-05-15 03:12:41.857596	127.0.0.1	1709a18f055730bb733a3144e415bea4dd81b...	3
5	logout	Cierre de sesión de Alex	2026-05-15 03:14:34.740143	127.0.0.1	d32b051fe63936c43e6849aa43a4f1cde7528ef...	3
6	login	Inicio de sesión de admin_prod	2026-05-15 03:15:05.653411	127.0.0.1	249a906347930926ab5ae51e9c5ed811defcad4...	1
7	verificacion_expediente	admin_prod verifiko 1 expediente(s)	2026-05-15 03:15:42.807437	127.0.0.1	88111b6cf53a2e468eb7355b5e1f0cf24e2bbd2...	1
8	logout	Cierre de sesión de admin_prod	2026-05-15 19:32:08.004612	127.0.0.1	57dd3ca6090ee10e8f66a91d0e4cbdd0e24871...	1
9	login	Inicio de sesión de admin_prod	2026-05-15 19:32:30.279011	127.0.0.1	f67c7a840001dc22cfbe6ffab8b8e59d5fe7c85d...	1
10	login	Inicio de sesión de admin_prod	2026-06-01 02:32:19.155611	127.0.0.1	0031518e796284257731e1dd5bd67bb229fcb4...	4
11	logout	Cierre de sesión de admin_prod	2026-06-01 02:35:18.856685	127.0.0.1	2c7699831bf58071c6d7d4e995f5e08791fbeb2...	4
12	login	Inicio de sesión de admin_prod	2026-06-01 15:49:34.494438	127.0.0.1	8c27a7592154e39e7c52e47a06a7b43c8f2ee15...	4
13	logout	Cierre de sesión de admin_prod	2026-06-01 19:18:40.539825	127.0.0.1	5fea45864f95051f7fca7e0d02f5f95b202ef664...	4
14	login	Inicio de sesión de admin_prod	2026-06-01 19:20:43.522068	127.0.0.1	e2528c85c5bfda266012d624bf1632fbb6b95f2e...	4
15	logout	Cierre de sesión de admin_prod	2026-06-01 19:57:00.338864	127.0.0.1	852489646a6ae799ddc8d010f8cc34121fee412...	4
16	login	Inicio de sesión de admin_prod	2026-06-01 19:57:17.024635	127.0.0.1	0a85799e4948271fd541f791c1a54579cdf6b25...	4

Figura 2: Bitácara de auditoría con encadenamiento de hashes SHA-256 (elaboración propia).

El acceso del portal del migrante funciona correctamente: folio y nombre correctos descifran el expediente; folio o nombre incorrectos producen el mismo mensaje genérico sin filtrar información sobre la existencia de folios; y el rate limit de cinco intentos por IP por 15 minutos bloquea correctamente intentos adicionales registrando el evento en la bitácara como `acceso_migrante_fallido`.

4.2 Resultados del Frontend

El sistema produce interfaces funcionales para todos los flujos de usuario identificados. El punto de entrada es la pantalla de inicio de sesión, mostrada en la Fig. 3, donde el usuario ingresa sus credenciales; al autenticarse, el backend desbloquea de forma transparente su llave privada RSA y las llaves de rol asociadas.

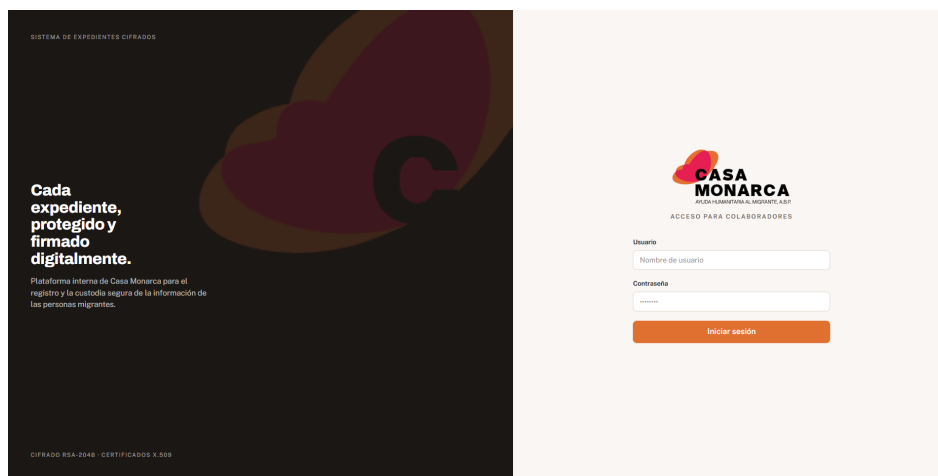


Figura 3: Pantalla de inicio de sesión del sistema .

Tras el login, el usuario llega al dashboard (Fig. 4), que adapta su contenido al rol activo: muestra los permisos disponibles, accesos rápidos a las funciones permitidas, badges de notificación para solicitudes pendientes y el estado de las llaves y el certificado del usuario.

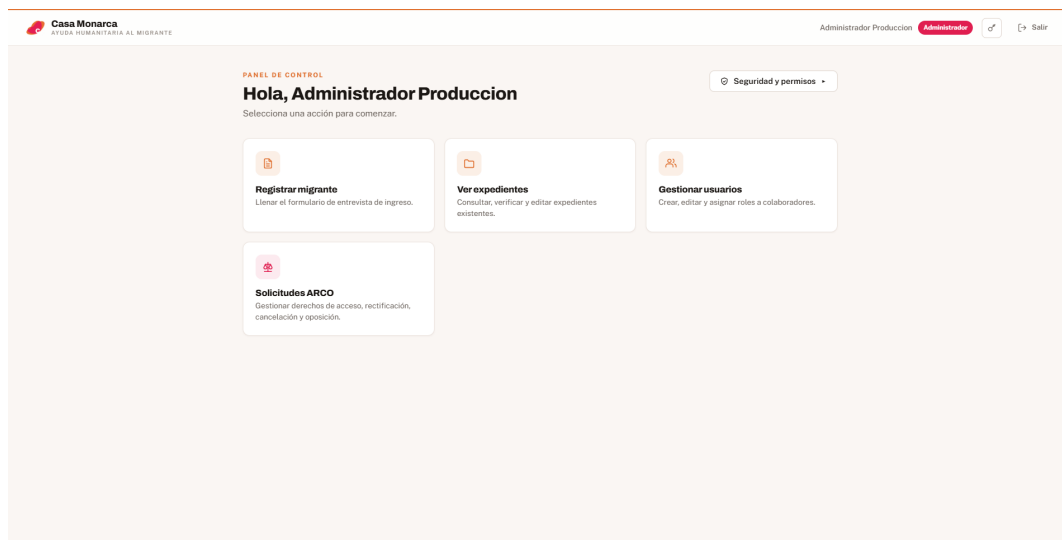


Figura 4: Dashboard adaptado al rol del usuario autenticado.

La lista de expedientes, mostrada en la Fig. 5, descifra y presenta los datos personales de los migrantes en tiempo real según las llaves de rol que el usuario posee en sesión, con filtros operativos por estado de verificación, creador y rango de fechas. Los expedientes para los que el usuario no tiene llave de rol se muestran como “Datos cifrados (sin acceso)”.

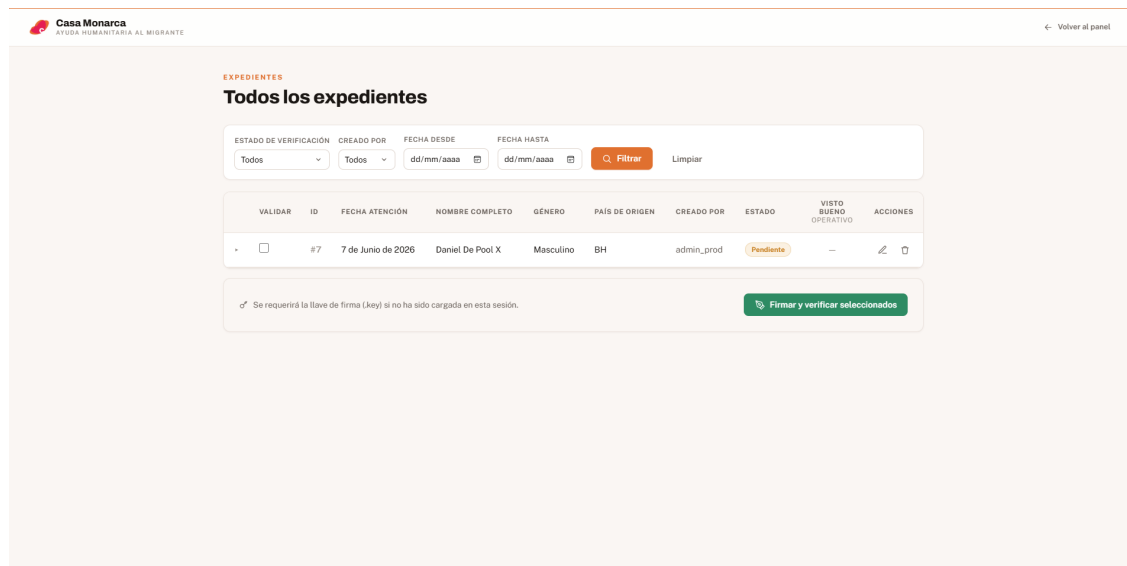


Figura 5: Lista de expedientes descifrados en tiempo real según el rol.

El Panel de Administración (Fig. 6) concentra la gestión de identidades en tres pestañas: Usuarios, Solicitudes de Rol y Bitácora. Desde aquí el Administrador crea usuarios, aprueba o rechaza solicitudes de cambio de rol, consulta la bitácora y regenera identidades criptográficas.

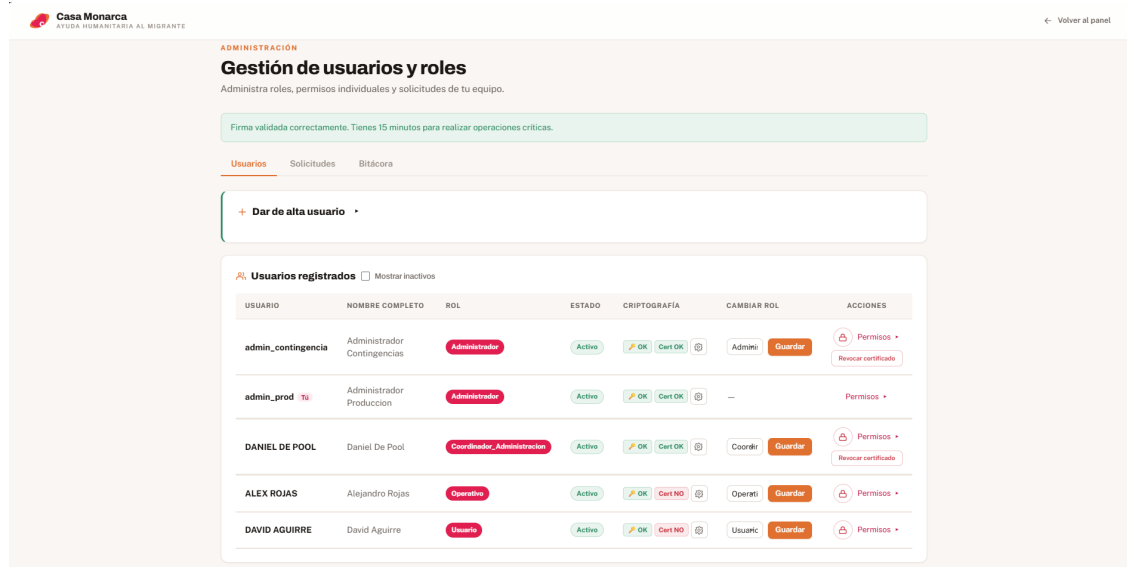


Figura 6: Panel de Administración con gestión de usuarios, roles y bitácora

Para las operaciones críticas, el sistema solicita la firma digital mediante la pantalla de la Fig. 7, donde el usuario sube su archivo `.key` e ingresa su passphrase. El sistema valida que el módulo de la llave privada coincida con el del certificado X.509 registrado, rechazando llaves que no correspondan al certificado activo.

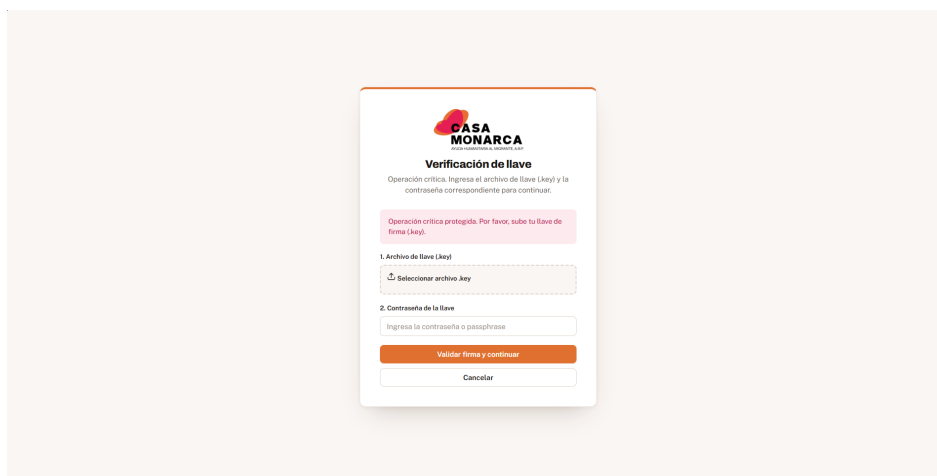


Figura 7: Pantalla de ingreso de firma digital mediante archivo `.key`.

Finalmente, el portal del migrante (Fig. 8) opera de forma independiente al sistema de autenticación Django, con una interfaz simplificada accesible sin cuenta de usuario, donde el migrante consulta sus datos y ejerce sus derechos ARCO usando solo su folio y nombre completo.

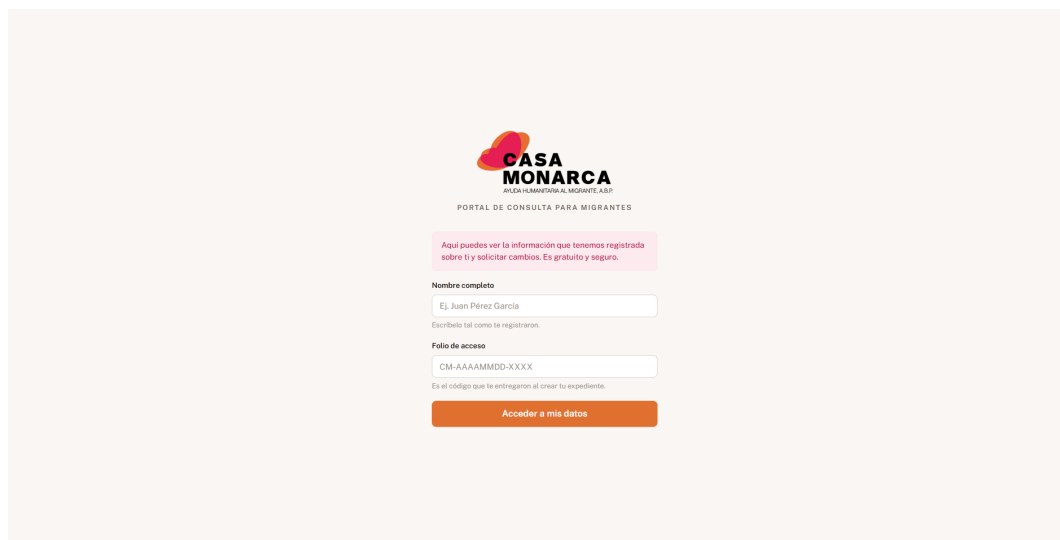


Figura 8: Portal público del migrante para consulta de datos y derechos ARCO .

4.3 Solución Completa: Flujo de Extremo a Extremo

Un recorrido completo del sistema ilustra la integración de todos los componentes: (1) Un Operativo se autentica; el sistema descifra su llave privada RSA con Scrypt(contraseña, salt) y carga la llave de rol Operativo en la sesión. (2) El Operativo registra un migrante; el sistema genera un folio único CM-YYYYMMDD-XXXX, cifra todos los datos con AES-256-EAX y una llave aleatoria de 256 bits, distribuye copias de la llave AES cifradas con RSA-OAEP para el creador y siete roles, y crea un acceso simétrico para el migrante vía Scrypt(folio:nombre). (3) El migrante accede al portal con su folio y nombre; el sistema deriva la clave, recupera la llave AES del AccesoExpediente tipo Migrante y descifra sus datos. (4) El migrante envía una solicitud de rectificación; el Operativo propone el diff JSON; el Coordinador ingresa su .key, firma la solicitud con RSA y aplica los cambios re-cifrando el expediente con la misma llave AES y actualizando el hash y la firma digital. (5) Todo el proceso queda registrado en la bitácora encadenada con SHA-256, con eventos firmados criptográficamente en los pasos que lo requieren.

4.4 Casos de Uso y Tabla de Validaciones

La Tabla 8 presenta los casos de prueba representativos de las ocho categorías validadas. Los archivos de evidencia se encuentran en el directorio `test_cases/` del repositorio del proyecto.

Tabla 8: Casos de prueba y validaciones del sistema (elaboración propia).

ID	Descripción	Rol	Resultado	Evidencia
<i>Categoría 01: Autenticación (13 casos)</i>				
TC-01-01	Login exitoso de Admin; desbloqueo de llaves RSA y llaves de rol en sesión	Admin	Aprobado	TC-01-01.md
TC-01-02	Login exitoso de Coordinador; módulo llave privada coincide con X.509	Coordinador	Aprobado	TC-01-02.md
TC-01-03	Login exitoso de Operativo; llave de rol Operativo en caché de sesión	Operativo	Aprobado	TC-01-03.md
TC-01-04	Login exitoso de Usuario; sin llave de rol (rol Usuario no tiene LlaveRol)	Usuario	Aprobado	TC-01-04.md
TC-01-05	Login con contraseña incorrecta; mensaje genérico sin revelar existencia del usuario	Todos	Aprobado	TC-01-05.md

Tabla 8 — continuación

ID	Descripción	Rol	Resultado	Evidencia
TC-01-07	Login con cuenta desactivada (activo=False); mensaje de cuenta desactivada	Todos	Aprobado	TC-01-07.md
TC-01-08	Login exitoso registra en BitacoraEvento: tipo, username e IP correctos	Todos	Aprobado	TC-01-08.md
TC-01-13	Login con llave privada corrupta; sesión cripto vacía; usuario no puede descifrar	Coordinador	Aprobado	TC-01-13.md
<i>Categoría 02: Gestión de Usuarios (22 casos)</i>				
TC-02-01	Crear Coordinador: genera RSA-2048, X.509 firmado por Admin, .key y ZIP	Admin	Aprobado	TC-02-01.md
TC-02-02	Crear Admin: distribuye TODAS las llaves de rol del sistema al nuevo Admin	Admin	Aprobado	TC-02-02.md
TC-02-05	Toggle activar/desactivar: campos activo e is_active cambian; bitácora registrada	Admin	Aprobado	TC-02-05.md
TC-02-06	Intentar desactivarse a sí mismo: mensaje de error, operación rechazada	Admin	Aprobado	TC-02-06.md
TC-02-07	Reset de contraseña: nuevas llaves RSA, nuevo salt, redistribución de llaves de rol	Admin	Aprobado	TC-02-07.md
TC-02-09	Cambio de contraseña propia: recifrado de llave privada con nuevo salt	Todos	Aprobado	TC-02-09.md
<i>Categoría 03: Roles y Permisos (20 casos)</i>				
TC-03-01	Cambio de rol a Coordinador: distribución de llave de rol; descifrado tras re-login	Admin	Aprobado	_index.md
TC-03-03	Cambio de rol a Admin: distribución de TODAS las llaves de rol	Admin	Aprobado	_index.md
TC-03-10	Solicitud de cambio de rol: creada con estado pendiente; registrada en bitácora	Usuario	Aprobado	_index.md
TC-03-15	Aprobación de solicitud: rol actualizado, llaves distribuidas, bitácora registrada	Admin	Aprobado	_index.md
<i>Categoría 04: Certificados y Firma Digital (22 casos)</i>				
TC-04-01	Ingresar firma con .key válido; llave_privada_firma guardada en sesión	Admin	Aprobado	_index.md
TC-04-03	Ingresar firma con passphrase incorrecta; mensaje de error sin revelar llave	Coordinador	Aprobado	_index.md
TC-04-04	Ingresar .key que no corresponde al certificado (módulus diferente); rechazo	Coordinador	Aprobado	_index.md
TC-04-11	Firma expirada (>15 min); operación crítica redirige a Ingresar Firma	Coordinador	Aprobado	_index.md

Tabla 8 — continuación

ID	Descripción	Rol	Resultado	Evidencia
TC-04-16	Middleware bloquea acceso con certificado X.509 expirado; logout forzado	Coordinador	Aprobado	_index.md
TC-04-22	X.509 generado muestra issuer=CN:admin y subject=CN:coordinador	Admin	Aprobado	_index.md
<i>Categoría 05: Expedientes CRUD (25 casos)</i>				
TC-05-01	Crear expediente: llave AES-256 aleatoria, AES-EAX, 9 filas AccesoExpediente	Usuario	Aprobado	_index.md
TC-05-06	Ver expedientes como Operativo: descifrado con llave de rol Operativo	Operativo	Aprobado	_index.md
TC-05-09	Ver expedientes como Usuario: solo muestra los expedientes creados por sí mismo	Usuario	Aprobado	_index.md
TC-05-19	Editar expediente: re-cifrado con misma llave AES, nuevo hash, firma, verificado=True	Coordinador	Aprobado	_index.md
TC-05-22	Eliminar expediente: borrado de expediente y AccesoExpediente; registro en bitácora	Admin	Aprobado	_index.md
TC-05-25	Sin llave de rol: expediente muestra "Datos cifrados (sin acceso)" en vez de PII	Cualquiera	Aprobado	_index.md
<i>Categoría 06: Verificación de Expedientes (10 casos)</i>				
TC-06-01	Verificar expediente: SHA-256 del ciphertext, firma RSA-PKCS1v15, verificado=True	Coordinador	Aprobado	_index.md
TC-06-02	Verificación batch: múltiples expedientes procesados; cantidad correcta en mensaje	Coordinador	Aprobado	_index.md
TC-06-05	Sin .key cargado: redirige a Ingresar Firma; tras ingresar, reanuda verificación	Coordinador	Aprobado	_index.md
<i>Categoría 07: Auditoría (19 casos)</i>				
TC-07-13	Encadenamiento de hashes: recalcular cadena SHA-256 coincide íntegramente con BD	Admin	Aprobado	_index.md
TC-07-14	Bloque génesis: primer registro usa literal 'GENESIS' como hash previo	Admin	Aprobado	_index.md
TC-07-15	Manipulación detectada: modificar un registro rompe la cadena en ese punto	Admin	Aprobado	_index.md
<i>Categoría 08: Seguridad Criptográfica</i>				
TC-08-01	Datos en BD no legibles: inspección directa de MySQL muestra solo ciphertext Base64	Admin	Aprobado	_index.md
TC-08-02	Rate limit portal migrante: 5 intentos fallidos bloquean IP por 15 minutos	Anónimo	Aprobado	_index.md

Tabla 8 — continuación

ID	Descripción	Rol	Resultado	Evidencia
TC-08-03	Mensaje genérico en portal: folio inexistente y nombre incorrecto dan el mismo error	Anónimo	Aprobado	_index.md

5. Conclusiones y Trabajo Futuro

5.1 Conclusiones

El Equipo 5 diseñó e implementó una plataforma de gestión segura para Casa Monarca que integra dos subsistemas complementarios y cumple íntegramente con los objetivos del reto. El **Gestor de Identidades** administra las credenciales criptográficas de los colaboradores —pares RSA-2048, certificados X.509 y archivos de firma— mientras que el **Gestor de Expedientes con Acceso por Roles** cifra los registros de los migrantes y controla criptográficamente quién puede descifrarlos. La separación entre ambos resultó clave: el primero responde por la identidad y el segundo por el acceso, y juntos garantizan que ningún dato sea legible sin la llave del rol correspondiente. La **Arquitectura de Seguridad de Doble Llave** separa además la autenticación de sesión —transparente, basada en Scrypt + AES-EAX— de la autorización criptográfica para operaciones críticas —explícita, basada en X.509 más archivo `.key`—, siguiendo el modelo de e.firma del SAT que resulta familiar y reconocible para el contexto mexicano.

El modelo de control de acceso granular por expediente, en el que la llave AES de cada registro se distribuye de forma independiente a cada rol mediante RSA-OAEP, garantiza que la confidencialidad de un expediente no depende de la secrecía de un único secreto maestro. Comprometer la llave de un rol no expone los expedientes accesibles por otros roles. La llave AES del expediente nunca se almacena en claro en ningún medio persistente: se genera en RAM, se usa para cifrar, se distribuye cifrada con RSA y se descarta.

La bitácora de auditoría con encadenamiento SHA-256 provee una garantía de integridad retrospectiva verificable: Casa Monarca puede en cualquier momento recalcular la cadena y confirmar que ningún registro ha sido alterado, eliminado o reordenado desde su creación. El portal del migrante implementa los derechos ARCO de forma criptográficamente sólida y sin exponer datos de otros migrantes, con un flujo de cancelación que requiere la firma independiente de un Coordinador y un Administrador, previniendo eliminaciones no autorizadas.

La decisión de implementar una solución on-premise usando exclusivamente bibliotecas de código abierto auditables (Pycryptodome y cryptography de PyCA) garantiza que Casa Monarca mantiene soberanía total sobre los datos de las personas que atiende, en cumplimiento con el espíritu de la LFPDPPP, sin dependencia de terceros y sin costos de licenciamiento recurrentes.

5.2 Trabajo Futuro

Las siguientes mejoras se identifican como prioritarias para etapas posteriores:

- **Autenticación de dos factores (2FA):** integrar TOTP (RFC 6238) como segundo factor en el flujo de login, especialmente para cuentas de Administrador, añadiendo una capa adicional de protección ante robo de contraseña.
- **Rotación periódica de SECRET_KEY:** implementar un comando de gestión Django que re-cifre todos los `EncryptedTextField` de la base de datos al rotar la `SECRET_KEY`, permitiendo cambios de clave de servidor sin pérdida de datos.
- **Despliegue con HTTPS:** configurar el servidor con certificado TLS para cifrar la comunicación en tránsito, complementando el cifrado en reposo ya implementado.
- **Exportación cifrada de expedientes:** implementar la funcionalidad de exportación que genere archivos PDF o ZIP con los datos del expediente cifrados, permitiendo compartirlos de forma segura entre sedes de Casa Monarca.
- **Panel de verificación de integridad de bitácora:** crear una vista en el Panel de Administración que recalcule y muestre el estado de la cadena de hashes, facilitando auditorías periódicas sin acceso directo a la base de datos.

Referencias

- Barker, E., Chen, L., Roginsky, A., Vassilev, A., Davis, R., y Simon, S. (2019). *NIST Special Publication 800-56B Revision 2: Recommendation for Pair-Wise Key Establishment Using Integer Factorization Cryptography* (NIST Special Publication). National Institute of Standards and Technology.
- Ramió Aguirre, J. (2006). *Libro electrónico de seguridad informática y criptografía* (4.1 ed.). (Capítulo 1: Presentación del Libro Electrónico)
- Shannon, C. E. (2001). A mathematical theory of communication (versión re-impresa). *ACM SIGMOBILE Mobile Computing and Communications Review*, 5(1), 3–55.

A. Flujo Criptográfico Completo del Sistema

A continuación se documenta la cadena de derivación de claves de extremo a extremo, desde la contraseña del usuario hasta los datos del migrante almacenados en MySQL.

Cadena de derivación de claves (Login Key):

1. Contraseña $\xrightarrow{\text{PBKDF2 Django}}$ campo password (autenticación estándar).
2. Contraseña $\xrightarrow{\text{Script(pwd, salt_login)}}$ clave_login_32B (AES-256, en memoria).
3. clave_login_32B $\xrightarrow{\text{AES-EAX decrypt}}$ llave_privada_RSA_usuario PEM (sesión Django).
4. llave_privada_RSA_usuario $\xrightarrow{\text{RSA-OAEP decrypt}}$ llave_privada_rol_X PEM (sesión Django, por cada rol asignado).
5. llave_privada_rol_X $\xrightarrow{\text{RSA-OAEP decrypt}}$ llave_aes_expediente (en memoria, por operación de lectura).
6. llave_aes_expediente $\xrightarrow{\text{AES-256-EAX decrypt}}$ JSON datos del migrante (en memoria, en respuesta HTTP).

Flujo de creación de expediente:

1. Backend genera llave_aes = os.urandom(32) (256 bits aleatorios).
2. Cifra datos: AES-EAX(llave_aes) → datos_cifrados, nonce, tag.
3. Calcula hash_expediente = SHA-256(datos_cifrados).
4. Calcula folio_hash = SHA-256(folio).
5. Para el creador: RSA-OAEP(pub_usuario, llave_aes) → AccesoExpediente(tipo='Creador').
6. Para cada rol: RSA-OAEP(pub_rol, llave_aes) → AccesoExpediente(tipo=nombre_rol).
7. Para el migrante: Script(folio:nombre, salt=SHA-256(folio)) → clave_folio; AES-EAX(clave_folio, llave_aes) → AccesoExpediente(tipo='Migrante').
8. Persiste Expediente en MySQL; descarta llave_aes de la RAM.

Flujo de firma de expediente (Signature Key):

1. @firma_requerida verifica que el timestamp de firma en sesión sea menor a 15 minutos.
2. Calcula hash_exp = SHA-256(datos_cifrados).
3. Firma: RSA-2048-PKCS1v15(llave_privada_firma, SHA-256(hash_exp)) → firma_b64.
4. Persiste Expediente.firma_digital = firma_b64, verificado = True.

B. Esquema Relacional de la Base de Datos

Las tablas centrales del sistema y sus relaciones clave son:

- **usuarios_usuario** ↔ **usuarios_accesollaverol** (1:N): un usuario tiene N accesos a llaves privadas de rol.
- **usuarios_llaverol** ↔ **usuarios_accesollaverol** (1:N): cada llave de rol está distribuida a N usuarios autorizados.
- **expediente_expediente** ↔ **expediente_accesoexpediente** (1:N): cada expediente tiene N accesos cifrados (creador + 7 roles + 1 migrante = hasta 9 filas).
- **usuarios_usuario** ↔ **expediente_expediente** (1:N): un usuario crea N expedientes.
- **expediente_expediente** ↔ **portal_migrante_solicitudarco** (1:N, SET_NULL al cancelar): las solicitudes ARCO conservan registro histórico tras borrar el expediente.
- **usuarios_usuario** ↔ **auditoria_bitacoraevento** (1:N, SET_NULL): los eventos del sistema se registran con usuario NULL para accesos del portal público.

La tabla **usuarios_accesollaverol** implementa el patrón de *envelope encryption*: la llave privada de cada rol se cifra individualmente con la llave pública RSA de cada usuario autorizado, de forma que cada usuario solo puede desempaquetar las llaves de sus roles asignados.

La tabla **expediente_accesoexpediente** implementa el mismo patrón para los expedientes: la llave AES de cada expediente se cifra con la llave pública de cada entidad autorizada. Esto permite revocar el acceso de un rol a todos los expedientes cambiando la llave RSA del rol y re-xcifrando sus filas de **AccesoExpediente**, sin necesidad de re-cifrar el contenido de los expedientes.